

are the same, $A(x)$ is necessarily the same as $B(x)$. (*Hint*: See Exercise 31.4-4.)

32.3 String matching with finite automata

Many string-matching algorithms build a finite automaton—a simple machine for processing information—that scans the text string T for all occurrences of the pattern P . This section presents a method for building such an automaton. These string-matching automata are efficient: they examine each text character *exactly once*, taking constant time per text character. The matching time used—after preprocessing the pattern to build the automaton—is therefore $\Theta(n)$. The time to build the automaton, however, can be large if Σ is large. Section 32.4 describes a clever way around this problem.

We begin this section with the definition of a finite automaton. We then examine a special string-matching automaton and show how to use it to find occurrences of a pattern in a text. Finally, we'll see how to construct the string-matching automaton for a given input pattern.

Finite automata

A *finite automaton* M , illustrated in Figure 32.5, is a 5-tuple $(Q, q_0, A, \Sigma, \delta)$, where

- Q is a finite set of *states*,
- $q_0 \in Q$ is the *start state*,
- $A \subseteq Q$ is a distinguished set of *accepting states*,
- Σ is a finite *input alphabet*,
- δ is a function from $Q \times \Sigma$ into Q , called the *transition function* of M .

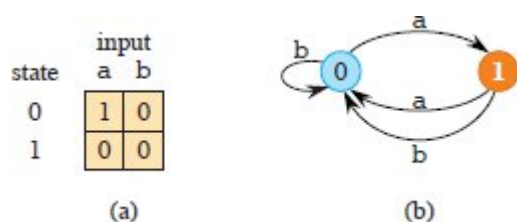


Figure 32.5 A simple two-state finite automaton with state set $Q = \{0, 1\}$, start state $q_0 = 0$, and input alphabet $\Sigma = \{a, b\}$. **(a)** A tabular representation of the transition function δ . **(b)** An equivalent state-transition diagram. State 1, in orange, is the only accepting state. Directed edges represent transitions. For example, the edge from state 1 to state 0 labeled b indicates that $\delta(1, b) = 0$. This automaton accepts those strings that end in an odd number of a's. More precisely, it accepts a string x if and only if $x = yz$, where $y = \epsilon$ or y ends with a b, and $z = a^k$, where k is odd. For example, on input $abaaaa$, including the start state, this automaton enters the sequence of states $\langle 0, 1, 0, 1, 0, 1 \rangle$, and so it accepts this input. For input $abbbaa$, it enters the sequence of states $\langle 0, 1, 0, 0, 1, 0 \rangle$, and so it rejects this input.

The finite automaton begins in state q_0 and reads the characters of its input string one at a time. If the automaton is in state q and reads input character a , it moves (“makes a transition”) from state q to state $\delta(q, a)$. Whenever its current state q is a member of A , the machine M has *accepted* the string read so far. An input that is not accepted is *rejected*.

A finite automaton M induces a function ϕ , called the *final-state function*, from Σ^* to Q such that $\phi(w)$ is the state M ends up in after reading the string w . Thus, M accepts a string w if and only if $\phi(w) \in A$. We define the function ϕ recursively, using the transition function:

$$\begin{aligned} \phi(\epsilon) &= q_0, \\ \phi(wa) &= \delta(\phi(w), a) \text{ for } w \in \Sigma^*, a \in \Sigma. \end{aligned}$$

String-matching automata

For a given pattern P , a preprocessing step constructs a string-matching automaton specific to P . The automaton then searches the text string for occurrences of P . Figure 32.6 illustrates the automaton for the pattern $P = ababaca$. From now on, let's assume that P is fixed, and

for brevity, we won't bother to indicate the dependence upon P in our notation.

In order to specify the string-matching automaton corresponding to a given pattern $P[1:m]$, we first define an auxiliary function σ , called the *suffix function* corresponding to the pattern P . The function σ maps Σ^* to $\{0, 1, \dots, m\}$ such that $\sigma(x)$ is the length of the longest prefix of P that is also a suffix of x :

$$\sigma(x) = \max \{k : P[1:k] \sqsupseteq x\} . \quad (32.3)$$

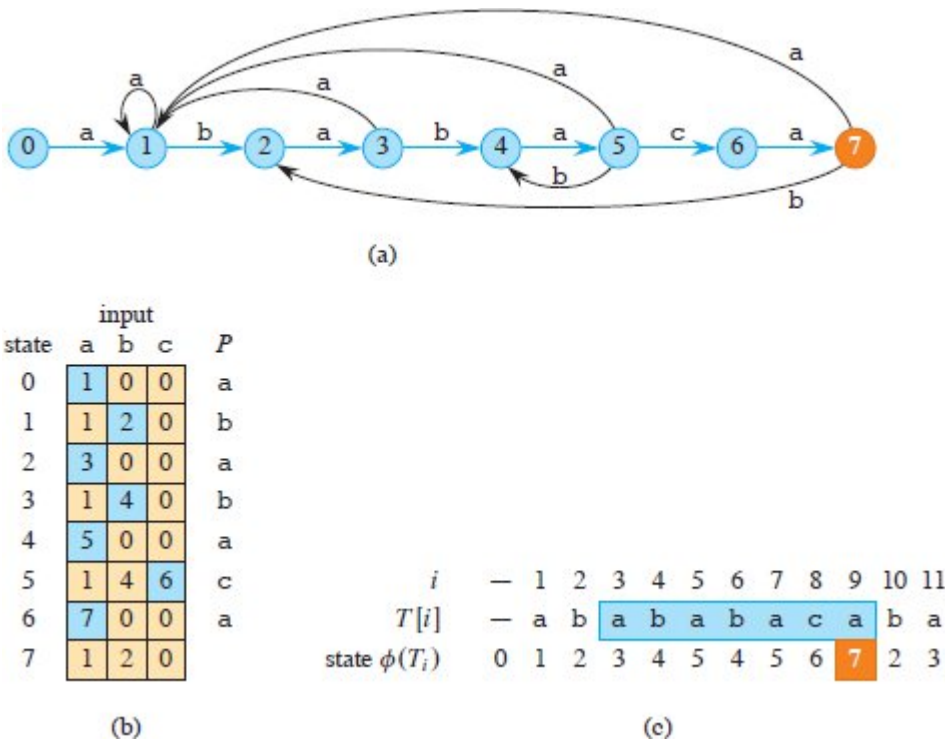


Figure 32.6 (a) A state-transition diagram for the string-matching automaton that accepts all strings ending in the string *ababaca*. State 0 is the start state, and state 7 (in orange) is the only accepting state. The transition function δ is defined by equation (32.4), and a directed edge from state i to state j labeled a represents $\delta(i, a) = j$. The right-going edges forming the “spine” of the automaton, shown in blue, correspond to successful matches between pattern and input characters. Except for the edges from state 7 to states 1 and 2, the left-going edges correspond to mismatches. Some edges corresponding to mismatches are omitted: by convention, if a state i has no outgoing edge labeled a for some $a \in \Sigma$, then $\delta(i, a) = 0$. (b) The corresponding transition function δ , and the pattern string $P = ababaca$. The entries corresponding to successful matches between pattern and input characters are shown in blue. (c) The operation of the automaton on the text $T = abababacaba$. Under each text character $T[i]$ appears the state $\phi(T[:i])$ that the automaton is in after processing the prefix $T[:i]$. The substring of the pattern that occurs in the text is highlighted in blue. The automaton finds this one occurrence of the pattern, ending in position 9.

The suffix function σ is well defined since the empty string $P[:0] = \epsilon$ is a suffix of every string. As examples, for the pattern $P = ab$, we have $\sigma(\epsilon) = 0$, $\sigma(ccaca) = 1$, and $\sigma(ccab) = 2$. For a pattern P of length m , we have $\sigma(x) = m$ if and only if $P \sqsupset x$. From the definition of the suffix function, $x \sqsupset y$ implies $\sigma(x) \leq \sigma(y)$ (see Exercise 32.3-4).

We are now ready to define the string-matching automaton that corresponds to a given pattern $P[1:m]$:

- The state set Q is $\{0, 1, \dots, m\}$. The start state q_0 is state 0, and state m is the only accepting state.
- The transition function δ is defined, for any state q and character a , by

$$\delta(q, a) = \sigma(P[:q]a) . \quad (32.4)$$

As the automaton consumes characters of the text T , it is trying to build a match of the pattern P against the most recently seen characters of T . At any time, the state number q gives the length of the longest prefix of P that matches the most recently seen text characters. Whenever the automaton reaches state m , the m most recently seen text characters match the first m characters of P . Since P has length m , reaching state m means that the m most recently seen text characters match the entire pattern, so that the automaton has found a match.

With this intuition behind the design of the automaton, here is the reasoning behind defining $\delta(q, a) = \sigma(P[:q]a)$. Suppose that the automaton is in state q after reading the first i characters of the text, that is, $q = \phi(T[:i])$. The intuitive idea then says that q also equals the length of the longest prefix of P that matches a suffix of $T[:i]$ or, equivalently, that $q = \sigma(T[:i])$. Thus, since $\phi(T[:i])$ and $\sigma(T[:i])$ both equal q , we will see (in Theorem 32.4 on page 973) that the automaton maintains the following invariant:

$$\phi(T[:i]) = \sigma(T[:i]) . \quad (32.5)$$

If the automaton is in state q and reads the next character $T[i + 1] = a$, then the transition should lead to the state corresponding to the longest prefix of P that is a suffix of $T[:i]a$. That state is $\sigma(T[:i]a)$, and equation (32.5) gives $\phi(T[:i]a) = \sigma(T[:i]a)$. Because $P[:q]$ is the longest prefix of P that is a suffix of $T[:i]$, the longest prefix of P that is a suffix of $T[:i]a$ has length not only $\sigma(T[:i]a)$, but also $\sigma(P[:q]a)$, and so $\phi(T[:i]a) = \sigma(P[:q]a)$. (Lemma 32.3 on page 972 will prove that $\sigma(T[:i]a) = \sigma(P[:q]a)$.) Thus, when the automaton is in state q , the transition function δ on character a should take the automaton to state $\delta(q, a) = \delta(\phi(T[:i]), a) = \phi(T[:i]a) = \sigma(P[:q]a)$ (with the last equality following from equation (32.5)).

There are two cases to consider, depending on whether the next character continues to match the pattern. In the first case, $a = P[q + 1]$, so that the character a continues to match the pattern. In this case, because $\delta(q, a) = q + 1$, the transition continues to go along the “spine” of the automaton (the blue edges in Figure 32.6(a)). In the second case, $a \neq P[q + 1]$, so that a does not extend the match being built. In this case, we need to find the longest prefix of P that is also a suffix of $T[1:q]a$, which will have length at most q . The preprocessing step matches the pattern against itself when creating the string-matching automaton, so that the transition function can quickly identify the longest such smaller prefix of P .

Let’s look at an example. Consider state 5 in the string-matching automaton of Figure 32.6. In state 5, the five most recently read characters of T are *ababa*, the characters along the spine of the automaton that reach state 5. If the next character of T is *c*, then the most recently read characters of T are *ababac*, which is the prefix of P with length 6. The automaton should continue along the spine to state 6. This is the first case, in which the match continues, and $\delta(5, c) = 6$. To illustrate the second case, suppose that in state 5, the next character of T is *b*, so the most recently read characters of T are *ababab*. Here, the longest prefix of P that matches the most recently read characters of T —that is, a suffix of the portion of T read so far—is *abab*, with length 4, so $\delta(5, b) = 4$.

To clarify the operation of a string-matching automaton, the simple and efficient procedure **FINITE-AUTOMATON-MATCHER** simulates the behavior of such an automaton (represented by its transition function δ) in finding occurrences of a pattern P of length m in an input text $T[1:n]$. As for any string-matching automaton for a pattern of length m , the state set Q is $\{0, 1, \dots, m\}$, the start state is 0, and the only accepting state is state m . From the simple loop structure of **FINITE-AUTOMATON-MATCHER**, you can see that its matching time on a text string of length n is $\Theta(n)$, assuming that each lookup of the transition function δ takes constant time. This matching time, however, does not include the preprocessing time required to compute the transition function. We address this problem later, after first proving

that the procedure `FINITE-AUTOMATON-MATCHER` operates correctly.

FINITE-AUTOMATON-MATCHER(T, δ, n, m)

```

1  $q = 0$ 
2 for  $i = 1$  to  $n$ 
3    $q = \delta(q, T[i])$ 
4   if  $q == m$ 
5     print "Pattern occurs with shift"  $i - m$ 

```

Let's examine how the automaton operates on an input text $T[1:n]$. We will prove that the automaton is in state $\sigma(T[i])$ after reading character $T[i]$. Since $\sigma(T[i]) = m$ if and only if $P \sqsupseteq T[i]$, the machine is in the accepting state m if and only if it has just read the pattern P . We start with two lemmas about the suffix function σ .

Lemma 32.2 (Suffix-function inequality)

For any string x and character a , we have $\sigma(xa) \leq \sigma(x) + 1$.

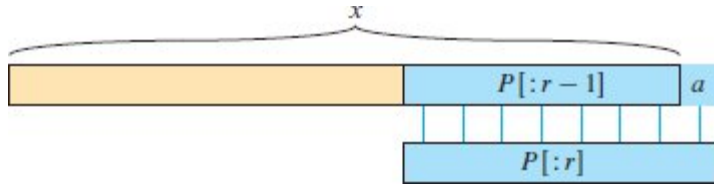


Figure 32.7 An illustration for the proof of Lemma 32.2. The figure shows that $r \leq \sigma(x) + 1$, where $r = \sigma(xa)$.

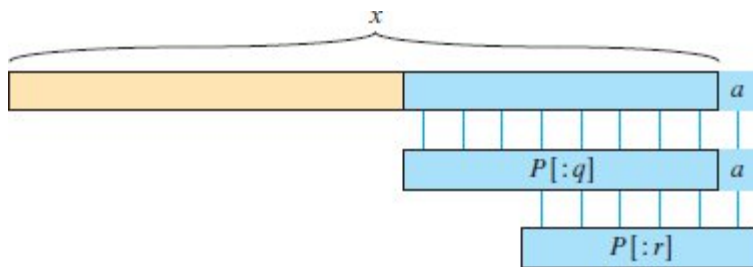


Figure 32.8 An illustration for the proof of Lemma 32.3. The figure shows that $r = \sigma(P[:q]a)$, where $q = \sigma(x)$ and $r = \sigma(xa)$.

Proof Referring to Figure 32.7, let $r = \sigma(xa)$. If $r = 0$, then the conclusion $\sigma(xa) = r \leq \sigma(x) + 1$ is trivially satisfied since $\sigma(x)$ is nonnegative. Now assume that $r > 0$. Then, $P[:r] \sqsupset xa$, by the definition of σ . Thus, $P[:r - 1] \sqsupset x$, by dropping the a from both the end of $P[:r]$ and the end of xa . Therefore, $r - 1 \leq \sigma(x)$, since $\sigma(x)$ is the largest k such that $P[:k] \sqsupset x$, and thus $\sigma(xa) = r \leq \sigma(x) + 1$. ■

Lemma 32.3 (Suffix-function recursion lemma)

For any string x and character a , if $q = \sigma(x)$, then $\sigma(xa) = \sigma(P[:q]a)$.

Proof The definition of σ gives that $P[:q] \sqsupset x$. As Figure 32.8 shows, we also have $P[:q]a \sqsupset xa$. Let $r = \sigma(xa)$. Then $P[:r] \sqsupset xa$ and, by Lemma 32.2, $r \leq q + 1$. Thus, we have $|P[:r]| = r \leq q + 1 = |P[:q]a|$. Since $P[:q]a \sqsupset xa$, $P[:r] \sqsupset xa$, and $|P[:r]| \leq |P[:q]a|$, Lemma 32.1 on page 959 implies that $P[:r] \sqsupset P[:q]a$. Therefore, $r \leq \sigma(P[:q]a)$, that is, $\sigma(xa) \leq \sigma(P[:q]a)$. But we also have $\sigma(P[:q]a) \leq \sigma(xa)$, since $P[:q]a \sqsupset xa$. Thus, $\sigma(xa) = \sigma(P[:q]a)$. ■

We are now ready to prove the main theorem characterizing the behavior of a string-matching automaton on a given input text. As noted above, this theorem shows that the automaton is merely keeping track, at each step, of the longest prefix of the pattern that is a suffix of what has been read so far. In other words, the automaton maintains the invariant (32.5).

Theorem 32.4

If ϕ is the final-state function of a string-matching automaton for a given pattern P and $T[1:n]$ is an input text for the automaton, then

$$\phi(T[:i]) = \sigma(T[:i])$$

for $i = 0, 1, \dots, n$.

Proof The proof is by induction on i . For $i = 0$, the theorem is trivially true, since $T[:0] = \varepsilon$. Thus, $\phi(T[:0]) = 0 = \sigma(T[:0])$.

Now assume that $\phi(T[:i]) = \sigma(T[:i])$. We will prove that $\phi(T[:i + 1]) = \sigma(T[:i + 1])$. Let q denote $\phi(T[:i])$, so that $q = \sigma(T[:i])$, and let a denote $T[i + 1]$.

+ 1]. Then,

$$\begin{aligned}
\phi(T[i+1]) &= \phi(T[i]a) && \text{(by the definitions of } T[i+1] \text{ and } a) \\
&= \delta(\phi(T[i]), a) && \text{(by the definition of } \phi) \\
&= \delta(q, a) && \text{(by the definition of } q) \\
&= \sigma(P[:q]a) && \text{(by the definition (32.4) of } \delta) \\
&= \sigma(T[i]a) && \text{(by Lemma 32.3)} \\
&= (T[i+1]) && \text{(by the definition of } T[i+1]).
\end{aligned}$$

■

By Theorem 32.4, if the machine enters state q on line 3, then q is the largest value such that $P[:q] \sqsupseteq T[:i]$. Thus, in line 4, $q = m$ if and only if the machine has just read an occurrence of the pattern P . Therefore, FINITE-AUTOMATON-MATCHER operates correctly.

Computing the transition function

The procedure COMPUTE-TRANSITION-FUNCTION on the following page computes the transition function δ from a given pattern $P[1:m]$. It computes $\delta(q, a)$ in a straightforward manner according to its definition in equation (32.4). The nested loops beginning on lines 1 and 2 consider all states q and all characters a , and lines 3–6 set $\delta(q, a)$ to be the largest k such that $P[:k] \sqsupseteq P[:q]a$. The code starts with the largest conceivable value of k , which is $q+1$, unless $q = m$, in which case k cannot be larger than m . It then decreases k until $P[:k]$ is a suffix of $P[:q]a$, which must eventually occur, since $P[:0] = \varepsilon$ is a suffix of every string.

COMPUTE-TRANSITION-FUNCTION(P, Σ, m)

```

1 for  $q = 0$  to  $m$ 
2   for each character  $a \in \Sigma$ 
3      $k = \min \{m, q + 1\}$ 
4     while  $P[:k]$  is not a suffix of  $P[:q]a$ 
5        $k = k - 1$ 
6      $\delta(q, a) = k$ 

```

7 return δ

The running time of COMPUTE-TRANSITION-FUNCTION is $O(m^3 |\Sigma|)$, because the outer loops contribute a factor of $m |\Sigma|$, the inner **while** loop can run at most $m + 1$ times, and the test for whether $P[:k]$ is a suffix of $P[:q]a$ on line 4 can require comparing up to m characters. Much faster procedures exist. By utilizing some cleverly computed information about the pattern P (see Exercise 32.4-8), the time required to compute δ from P improves to $O(m |\Sigma|)$. This improved procedure for computing δ provides a way to find all occurrences of a length- m pattern in a length- n text over an alphabet Σ with $O(m |\Sigma|)$ preprocessing time and $\Theta(n)$ matching time.

Exercises

32.3-1

Draw a state-transition diagram for the string-matching automaton for the pattern $P = \text{aabab}$ over the alphabet $\Sigma = \{a, b\}$ and illustrate its operation on the text string $T = \text{aabababaabaababab}$.

32.3-2

Draw a state-transition diagram for the string-matching automaton for the pattern $P = \text{ababbabbababbababbabb}$ over the alphabet $\Sigma = \{a, b\}$.

32.3-3

A pattern P is *nonoverlappable* if $P[:k] \sqsupset P[:q]$ implies $k = 0$ or $k = q$. Describe the state-transition diagram of the string-matching automaton for a nonoverlappable pattern.

32.3-4

Let x and y be prefixes of the pattern P . Prove that $x \sqsupset y$ implies $\sigma(x) \leq \sigma(y)$.

★ 32.3-5

Given two patterns P and P' , describe how to construct a finite automaton that determines all occurrences of *either* pattern. Try to minimize the number of states in your automaton.

32.3-6

Given a pattern P containing gap characters (see Exercise 32.1-4), show how to build a finite automaton that can find an occurrence of P in a text T in $O(n)$ matching time, where $n = |T|$.

★ 32.4 The Knuth-Morris-Pratt algorithm

Knuth, Morris, and Pratt developed a linear-time string matching algorithm that avoids computing the transition function δ altogether. Instead, the KMP algorithm uses an auxiliary function π , which it precomputes from the pattern in $\Theta(m)$ time and stores in an array $\pi[1:m]$. The array π allows the algorithm to compute the transition function δ efficiently (in an amortized sense) “on the fly” as needed. Loosely speaking, for any state $q = 0, 1, \dots, m$ and any character $a \in \Sigma$, the value $\pi[q]$ contains the information needed to compute $\delta(q, a)$ but that does not depend on a . Since the array π has only m entries, whereas δ has $\Theta(m |\Sigma|)$ entries, the KMP algorithm saves a factor of $|\Sigma|$ in the preprocessing time by computing π rather than δ . Like the procedure `FINITE-AUTOMATON-MATCHER`, once preprocessing has completed, the KMP algorithm uses $\Theta(n)$ matching time.

The prefix function for a pattern

The prefix function π for a pattern encapsulates knowledge about how the pattern matches against shifts of itself. The KMP algorithm takes advantage of this information to avoid testing useless shifts in the naive pattern-matching algorithm and to avoid precomputing the full transition function δ for a string-matching automaton.

Consider the operation of the naive string matcher. Figure 32.9(a) shows a particular shift s of a template containing the pattern $P = \text{ababaca}$ against a text T . For this example, $q = 5$ of the characters have matched successfully, but the 6th pattern character fails to match